Torneo de Komikan Concurrente

Informe de Diseño y Sincronización

Universidad Diego Portales – Escuela de Informática y Telecomunicaciones
Por: Javier Santibáñez
Fecha: 03 de noviembre de 2026

1. Link del Github:

`https://github.com/TumareCh4n/Sistemas-Operativos-Tarea-II`

1. Diseño del esquema de sincronización

Se implementó un torneo concurrente en C++17 utilizando hilos POSIX (`pthread`), mutexes y variables de condición. La arquitectura se divide en tres componentes principales: *Matchmaker*, *Mesa* y *Mapa*.

Matchmaker: Administra la llegada de N jugadores mediante hilos concurrentes, emparejándolos en grupos de dos. Los jugadores disponibles se almacenan en una cola FIFO protegida por un mutex. Se crean hasta M trabajadores de mesa, reutilizados al finalizar cada partida.

Cuando no existen suficientes jugadores para formar una pareja, los trabajadores utilizan `pthread_cond_wait()`, evitando la espera activa. De esta forma, el número de partidas simultáneas nunca supera el límite establecido.

Mesa y Juez: Cada mesa posee un tablero independiente, trece hilos correspondientes al Puma y los doce Perros, y un hilo Juez encargado de supervisar el desarrollo de la partida.

El Juez inicializa las posiciones, registra periódicamente el tablero, detecta condiciones de victoria y coordina la finalización de los hilos mediante `pthread_join()`.

Exclusión mutua: El tablero compartido se protege mediante un único mutex denominado `expansionDeDominio`. Cada ficha debe adquirirlo antes de consultar posiciones, seleccionar un movimiento y modificar el tablero.

La operación completa se ejecuta dentro de una sección crítica, garantizando que dos fichas no puedan ocupar simultáneamente una misma posición. Al terminar, se libera el mutex.

Los movimientos se realizan de forma independiente con retardos aleatorios configurables. El Juez recibe notificaciones mediante una variable de condición y consulta el estado sin interferir con las modificaciones del tablero.

2. Análisis de riesgos de concurrencia

Se identificaron los siguientes riesgos y sus respectivas medidas de prevención:

- **Race condition:** Dos fichas pueden intentar acceder simultáneamente a una misma posición. Se evita mediante exclusión mutua durante la validación y actualización completa del movimiento.
- **Deadlock:** Podría producirse por adquisición circular de recursos. Cada tablero utiliza un único mutex para sus operaciones, evitando adquirir múltiples bloqueos de casillas. Las variables de condición liberan el mutex mientras esperan.
- **Starvation:** Una ficha podría recibir menos oportunidades de movimiento debido a la planificación de los hilos. Los retardos aleatorios reducen la competencia constante, aunque no garantizan equidad.
- **Livelock:** Los movimientos aleatorios pueden producir desplazamientos repetitivos sin progreso hacia una victoria. La sincronización mantiene la consistencia del tablero, pero no garantiza que una partida finalice rápidamente.

Para la interrupción mediante `SIGINT`, se utiliza un hilo vigilante con `sigwait()` y una bandera atómica `zaWarudo`. Al recibir la señal, se detienen las nuevas inscripciones y se solicita el cierre ordenado de las mesas. Los hilos son reunidos antes de destruir los recursos POSIX.

3. Evaluación y decisiones de diseño

Se escogió un mutex global por tablero en lugar de bloqueos individuales por casilla, priorizando la simplicidad y consistencia del sistema. Esta decisión reduce la complejidad de implementación y evita problemas de adquisición de múltiples recursos, a cambio de permitir solamente un movimiento crítico a la vez por mesa.

Las diferentes mesas pueden ejecutarse simultáneamente, ya que cada una mantiene su propio tablero y primitivas de sincronización. El Matchmaker limita la concurrencia a M mesas, mientras la espera pasiva evita consumir CPU innecesariamente.

Se incorporó un archivo de configuración para modificar retardos, semilla aleatoria y posiciones iniciales. La semilla permite repetir las secuencias de decisiones por hilo, aunque el orden de ejecución puede variar debido a la planificación del sistema operativo.

Las pruebas funcionales consideran distintas cantidades de jugadores y mesas, movimientos concurrentes, capturas, emparejamiento, configuraciones inválidas e interrupciones del servidor.

Limitaciones: La simulación de abandono permite verificar la resolución por retiro de un jugador, pero no equivale a detectar cualquier fallo inesperado de un hilo. Asimismo, la estrategia aleatoria no garantiza un tiempo finito de partida y los mensajes de distintas mesas pueden entremezclarse en consola.

En conjunto, la arquitectura favorece la seguridad del estado compartido, la reutilización de recursos y una implementación modular, manteniendo un equilibrio razonable entre complejidad y concurrencia.